
SCHOOL OF ENGINEERING AND TECHNOLOGY

FINAL ASSESSMENT FOR BSC (HONS) IN COMPUTER SCIENCE; BSC (HONS) INFORMATION TECHNOLOGY; BACHELOR OF SOFTWARE ENGINEERING (HONS); BSC (HONS) INFORMATION SYSTEMS (DATA ANALYTICS)

YEAR 2; ACADEMIC SESSION SEPTEMBER 2026

SEG2102 DATABASE MANAGEMENT SYSTEMS

DUE DATE: BY END OF THE 10 JANUARY (SATURDAY), 2026

INSTRUCTIONS

- This final assessment contributes **30% to final grade**.
- This **three-member (3-member) group assignment** is primarily for Course Learning Outcomes 1 and 3 - Examine the advanced concept of different database models concerning their design, strengths, and weaknesses from the aspects of models' design, data integrity, database scalability, and access control.
- **Submission:** Submit a **single MS Word file** as your report. This assessment requires GROUP submission through the **TURNITIN** system, meaning GROUP LEADER must submit their own report based on the group work via **TURNITIN**.
- **Report Format:** Prepare a report using the **IEEE template**.
- **Detailed Requirements:** For specific details about the assignment, please refer to pages 2-3.
- **Plagiarism Policy:** Turnitin report must indicate **less than 20% similarity**, including AI-generated content.

IMPORTANT

The University requires students to adhere to submission deadlines for any form of assessment. Penalties are applied in relation to unauthorized late submission of work.

Academic Honesty Acknowledgement

"We Wong Hui San, Keertana A/P Subramaniam, Liu Yong Le verify that this paper contains entirely our own work. We have not consulted with any outside person other than what was specified in the assignment requirements. Further, we have not copied or inadvertently copied ideas, sentences, or paragraphs from another student. We realize the penalties (*refer to the student handbook of undergraduate programme*) for any kind of copying or collaboration on any assignment."

Signature:	<u>huisan</u>	<u>keertana</u>	<u>yongle</u>
Student ID:	22034540	23109614	22064141
Name:	Wong Hui San	Keertana A/P Subramaniam	Liu Yong Le
Date:	28/11/2025	28/11/2025	28/11/2025

A Study of Query Optimization for Complex Join Queries Using Causal-Guided Reinforcement Learning

1st Wong Hui San

School of Computing and Artificial Intelligence

No. 5, Jalan Universiti, Bandar Sunway, 47500, Selangor Darul Ehsan, Malaysia
22034540@imail.sunway.edu.my

2nd Keertana A/P Subramaniam

School of Computing and Artificial Intelligence

No. 5, Jalan Universiti, Bandar Sunway, 47500, Selangor Darul Ehsan, Malaysia
23109614@imail.sunway.edu.my

3rd Liu Yong Le

School of Computing and Artificial Intelligence

No. 5, Jalan Universiti, Bandar Sunway, 47500, Selangor Darul Ehsan, Malaysia
22064141@imail.sunway.edu.my

Abstract— Query optimization is one of the important features in Database Management Systems (DBMS). It directly influences the performance and implementation of tools. Traditional optimizers often struggle to deliver an efficient plan due to issues such as inaccurate estimation and large search spaces. This study have reviewed different advanced query optimization strategies that focus on reinforcement learning approaches. The goal is to examine if adaptive learning can perform better than supervised models in improving join order for complex queries. Although the existing machine learning models have better accuracy, they are lack of adaptability and frequently fail under complex workloads. It leads to high execution costs and poor adaptability due to insufficient decisions for multi-join queries. The report used a comparative study to compare the proposed CR-GL with other supervised models such as Decision Tree, Random Forest, and Logistic Regression. The results show that supervised models perform better in short-term accuracy. However, CR-GL showed a strong learning ability in long-term situations. These findings highlight the need for significant training time and hardware requirements in CR-GL to perform better in accuracy and efficiency.

Keywords— Query Optimization, Reinforcement Learning, CR-GL, Multi-Join Queries, Database Management Systems, Performance

I. INTRODUCTION

In the world where data is scattered around everywhere, databases are important as it is used to store and organize information so that it is easier to find the required information later. Database Management System (DBMS) is a set of programs that help to generate, store, retrieve and manage the data stored effectively. It acts as a bridge between databases and end-users and uses query languages to communicate with the database to understand the requests [1]. Therefore, query processing and optimization are crucial components in a DBMS as it will influence the both the performance and the quality of the results returned to users. Query processing helps to understand and translate user queries into a machine-readable form, then executes them using the quickest or most optimised method to retrieve the required information [2].

A. Problem Statement

Most of the system uses traditional optimizer called cost-based optimizer. However, cost-based optimizer can make mistakes and became less efficient when dealing with

complex join queries. This is because the hidden correlations between columns in different tables makes it difficult to accurately estimate the cardinality. Approximations such as the inclusion principle is used but it can produce large errors if assumption fail [3]. For example, Marcus et al. [4] state that PostgreSQL optimizer sometimes underestimate the cardinality for certain joins query and choose loop join to execute the query, even when other join algorithms like merge join and hash join would be more efficient.

On the other hand, when a query involves many tables, the optimizer may use heuristics algorithms or shortcuts to generate a query plan and avoid wasting too much time finding the best plan. As a result, the chosen plan may be suboptimal, leading to slower or less efficient query execution. For example, the genetic algorithm in PostgreSQL and the greedy algorithm in DB2 apply heuristics to reduce optimization time [3].

B. Significance of the Problem

Addressing the inefficiency of query optimizers in handling complex joins is important because the need to process large and complex queries and is increasing in this modern world. A single analytical query in business environments may contain up to a few hundreds of tables [5]. If the optimizer selects a suboptimal execution plan, query processing can take much longer affecting the overall performance as there are too many queries still waiting to be processed. An efficient query optimizer ensures that even complex queries are executed quickly, improving performance and resource utilization.

C. Objective

The objective of this paper is to analyze and evaluate existing query optimization techniques for handling complex multi-join queries. This paper proposes a novel solution using reinforcement learning guided by causal relationships. The proposed solution is compared with existing techniques by evaluating their performance across different metrics such as the query execution efficiency, learning capability and join selection quality. The objective is to identify the strengths, limitations, and the effectiveness of different query optimization techniques when handling complex joins.

D. Scope of the Report

This report focuses on analyzing existing query optimization techniques for complex multi-join queries by reviewing previous studies, examining the methods they use and the performance results they reported. The study involves implementing a novel reinforcement learning-based optimization approach guided by causal relationships. It is compared with selected baseline techniques to assess their strengths, weaknesses and suitability for different workload conditions. The scope is restricted to experimental evaluation with simulated relational database settings, and a supplementary observational analysis of NoSQL query execution behavior in MongoDB. It does not cover large-scale real-world production databases or non-relational database systems.

II. LITERATURE REVIEW

A. Structure of the Literature Review

1) Introduction

This literature review explores recent advancements in query optimization within Database Management Systems (DBMS) with a focus in the difficulties presented by intricate multi-join queries. Queries involving multiple tables have become more prevalent as modern applications produce increasingly larger and more interconnected datasets, especially in analytical workloads and decision-support systems. The ability of DBMS's optimizer to produce effective execution plans is crucial to its performance, yet problems such as inaccurate cardinality prediction, hidden attribute correlations, and the vast search space of possible join orders continue to limit accuracy. Heuristics are frequently used by traditional cost-based optimizers to reduce planning time, which can result in not very ideal plans and slower query performance.

This section highlights the groundwork for assessing existing techniques and proposing improvements in managing intricate join queries by reviewing recent research and optimization strategies and highlighting key trends, constraints, and emerging solutions.

2) Selection Criteria

The reviewed articles were chosen based on their applicability to multi-join query optimization in relational and distributed database systems. Studies published between 2021 and 2025 that addressed issues with traditional query optimizers, suggested machine learning or AI-driven optimization techniques, or investigated heuristic, combinatorial, and GPU-based methods for increasing query execution efficiency were the focus of the inclusion criteria. Only peer-reviewed journal articles, conference papers, and high-quality preprints were considered for this literature review. Studies were further filtered to guarantee a focus on useful enhancements in cardinality estimation, join order selection, execution plan quality, and performance under dynamic or large-scale workloads, studies were further filtered.

3) Organization

This review is structured into five thematic sections that represents the main areas of the modern query optimization

research. To overcome the limitation in traditional optimizers, especially for intricate multi-join queries and dynamic workloads, the first section concentrates on AI-driven and machine learning (ML) based optimizers that integrate reinforcement learning, neural models, bandit strategies and hybrid cost predictors. In the next section, optimization approaches which are evolutionary, metaheuristic, and combinatorial are examined. The third section examines development in cardinality estimation and resents sketch-based structures and ML-based estimators to lessen the errors that often result in ineffective plans. The next section explores the adaptive and runtime optimization techniques which modify query execution based on real-time system feedback. This enables the databases to respond to workload variability and enhance robustness without requiring extensive training. The fifth and final section focuses on hardware accelerated optimization with a focus on GPU-based methods that use parallel processing to greatly accelerate join enumeration and execution. These themes together offer an organized understanding of how learning-based, heuristic, probabilistic, adaptive, and hardware-supported methods have been used in recent research to enhance the performance of query.

B. Review of Articles

1) AI-Driven and Machine Learning-Based Query Optimizer

Many recent journal articles or papers have discussed about using AI-driven and machine learning-based optimizers to improve the efficiency of traditional query optimizers. The papers mentioned that traditional query optimizers often struggle with dynamic workloads, complex multi-join queries, inaccurate cardinality estimation and large search space. Therefore, the authors aimed to address these issues using AI and machine learning approaches to enhance the performance.

Sassi and Jaziri [6] propose a hybrid improvement framework called Graph-based Reinforcement Learning Query Optimizer (GRQO) to combines reinforcement learning with graph-based schema representations. The approach employs proximal policy improvement for adaptive learning and graph theory to improve join order selection and cardinality estimation. Experimental results show that GRQO reduces query execution time by 25% and improves adaptability by 30%. Besides that, it also improves resource efficiency compared to conventional cost-based and machine learning-based optimizers.

Panwar [7] studied on the AI-driven query optimization techniques that use machine learning models to predict and perform efficient query plans using history and real-time data. The findings show that AI-based optimizers are performing better than conventional methods in performance, flexibility, and efficiency. This highlights the potential for automated database systems.

Bao et al. [4] stated that previous machine learning techniques proposed by other studies are not practical because they needed long training time and require retraining when data changes. Therefore, they proposed a

learned optimizer called Bao (Bandit Optimizer) that uses both machine learning and bandit techniques. For every incoming SQL query, Bao will generate different query plans using different hint sets. Then, the performance of each set of plans is predicted using a tree-convolution neural network (TCNN). Thompson Sampling is used to choose the plan that balances both exploration and exploitation. Model is retrained periodically using the recorded actual performance of the plans. Bao performs almost 50% better than PostgreSQL in cloud cost and execution time by reducing the tail latency. When handling query involve joins, Bao seldom causes regressions and some queries improved by 20+ seconds.

Karri [8] proposed an AI-based method to improve query optimization. It consists of three AI modules which are Learned Cardinality Estimator (LCE), Neural Cost Corrector (NCC) and Reinforcement-Learning Planner (RLP). LCE predicts how many rows a query will return, NCC fixes errors in the cost estimates and RLP suggests the best order to join tables. The system constantly collects data from the query execution and uses this feedback to improve AI models. The AI optimizer has improved performance especially in handling complex queries with correlated predicates and increases accuracy by ~11% and F1 score by ~9%. It also reduces the execution time by approximately 40% on average particularly for multi-join queries. Furthermore, it reduces CPU and memory usage by 25% to 30% and maintain its performance even under high query loads.

Messaoudi and Touahria [9] suggested a machine learning-based method that depicts each join plan as a feature vector that captures base relation statistics, physical operator choices and structural characteristics of the join graph. Linear and piecewise linear models are trained on past execution logs to predict plan costs and intermediate cardinalities. These models then integrated into the optimizer to guide pruning decisions and enhance selection of plan. This is all done while preserving deterministic behaviour of conventional techniques. The study mentions that these learning-based models provide a scalable framework for relational databases that are large-scale, enable effective evaluation within the optimizer, and improve the accuracy of cost and cardinality estimation for complex multi-join queries. These findings emphasize that incorporating data-driven models with traditional optimization techniques can greatly enhance performance of the query and plan quality.

The study by Ji et al. [10] addressed the issue of conventional optimization techniques difficulty in efficiently explore the vast search space. The study suggested a dynamic double deep Q-network (DDQN) approach for join order optimization (DDOS) to overcome these limitations. It was done by modelling queries as a Markov decision process and incorporating DDQN weighting to lower overestimation errors in action value estimation. To improve exploration depth and randomness, a dynamic progressive search was employed. The results showed that the proposed approach outperforms conventional dynamic programming, heuristic algorithms,

and standard deep reinforcement learning techniques. It also greatly enhances performance of query and generalizes well to new queries.

Karri and Muntala [11] wanted to improve three key areas of optimization by addressing the issue of inaccurate cardinality estimates and suboptimal plans for complex multi-join queries using machine-learning based approach. First is to predict cardinalities and costs. Next, it makes decisions on join ordering and operator selection using reinforcement learning. Lastly, it controls execution with bandit-based online adaptation. Experiments on analytical and mixed workload demonstrate that this method reduces execution time and improves reliability of plan. It also addresses challenges such as cold starts, query log privacy, and integration with concurrency control. The study suggests future improvements in cross-database transfer and explainability.

2) Evolutionary, Metaheuristic and Combinatorial Optimization

Besides using machine learning or AI methods that learn patterns from data to make decision, there are other methods that are simpler to implement like heuristic or combinatorial methods. These methods use predefined rules, search strategies or optimization techniques to explore possible solutions efficiently. Several studies have shown the effectiveness of these methods.

Du et al. [12] focused their studies on improving join queries in distributed databases. The significant challenges are such as CPU computation, I/O cost and network delay. The authors proposed a development system for that combines genetic algorithms, artificial bee colony strategies, and spectral clustering. By using this algorithm, the development can be faster and enhance flexibility in performing plan search. In addition, the results show that this solution makes it suitable for complex joins across multiple nodes. It is because the query response time and execution costs are lesser than traditional optimization techniques.

Gaikwad et al. [13] study combined optimization approaches to improve query performance. The authors have studied complex multi-join queries where detailed description is complex. The study has reviewed algorithms such as genetic algorithms, simulated annealing, and tabu search. These algorithms can access a huge query limit efficiently to find a practical solution. Furthermore, the results show that these heuristic-based strategies can perform better than traditional methods. It is because these strategies can avoid local optima and improve global query performance. Besides that, these strategies allowed users to handle complex join operations effectively by enhancing cost-based optimization methods.

Karthikeyan et al. [14] proposed utilizing reinforcement learning (Q-Learning) and Ant Colony Optimization (ACO) algorithms to enhance query execution efficiency in distributed database environments. The suggested techniques dynamically modify execution plans to reduce query costs and workload delays. The experimental results emphasized that both Q-Learning and ACO techniques enhanced query optimization by identifying optimal join

sequences, reducing execution costs, and successfully managing constantly changing data environments. The study emphasizes how metaheuristic optimization and reinforcement learning can be combined to improve multi-join query performance in relational database.

3) Cardinality Estimation Improvements

One of the major issues in traditional query optimiser that causes inefficiency is the inaccurate cardinality estimation. Poor estimates of intermediate result sizes can cause optimizer to select inefficient query plans that impacts execution time and resource usage. That is why multiple studies specifically target the root cause of poor plans instead of proposing solution on join search algorithms.

The study by Justen and Boehm [15] identified that traditional cost-based optimizers often create poor query plans for multi-join queries due to inaccurate cardinality estimates. This problem is especially obvious in multi-predicate or complex join graph. Although learning-based optimizers exist, they face high training costs, limited generalization and challenges with integration. The authors suggest OmniSketch which is a probabilistic data structure that combines count-min sketches and K-miniwise hashing. This structure estimates multi-join cardinalities without relying on uniformity or independence. OmniSketch was introduced to join estimators that work with multiple tables and provide an algorithm for alpha-acyclic join graphs. Experiments were carried out on SSB-skew and JOB-light workloads. OmniSketch improves estimation accuracy and quality of query plan compared to traditional optimizers such as DuckDB. It reduces the sizes of result by up to 1,077 times and cuts the time of execution by up to 3.19 times in SSB-skew. JOB-light workloads showed occasional improvements but struggles with poor join graph shapes and large foreign-key value sets. This emphasizes the probabilistic sketch-based methods effectiveness for estimating multi-join cardinality.

Han et al. [16] also reviewed that although many advanced machine learning-based cardinality estimation methods have been developed, there is limited evidence on how much they can enhance query optimizer performance in real-world database management system settings. Common metrics such as Q-Error may not accurately reflect the overall query plan quality. The authors created a new benchmark using a complex real-world dataset (STATS) and a diverse query workload (STATS-CEB). They incorporated several representative cardinality estimation methods into PostgreSQL and assess their effects on quality of query plan, speed of inference, time of training, update efficiency, and overall usability. They also proposed a new metric, P-Error, to better evaluate cardinality estimation performance during end-to-end query execution. The study demonstrates that high estimation accuracy alone does not guarantee improved query plans. Better reflection was provided by P-Error of how cardinality estimation impacts the actual performance of query. Their evaluation offers insights into the trade-offs between accuracy, speed, and efficiency for machine learning-based cardinality estimation methods.

4) Adaptive & Runtime Optimization

Adaptive and runtime optimization techniques allow database to adjust its query execution while the query is running, which means, it can respond to sudden changes in data and modify its execution plan accordingly. These techniques are different with machine learning approaches, they use lightweight and practical strategies to make databases more stable, faster and better at handling real-world workloads.

Zhang et al. [17] compare reinforcement learning-based query optimizers with lightweight adaptive query processing techniques. The study implements two adaptive strategies in PostgreSQL which are dynamic switching between join algorithms (Nested Loop and Hash Join) and Lookahead Information Passing (LIP) for efficient join pipelines. The results show that these simple adaptive methods often outperform RL-based optimizers and offered competitive performance without expensive training steps. The authors emphasize interpretability and robustness to encourage for the use of practical adaptive techniques as an alternative to complex machine learning-based solutions.

Rodriguez and Turner [18] addressed the limitations of traditional optimization techniques in managing the “3 Vs” of big data which are Volume, Velocity, and Variety. The authors also examine advanced approaches such as distributed query execution, parallel processing, and machine learning-based optimization. They used it to conclude that hybrid frameworks combining cost-based and AI-driven methods deliver the most promising results. Furthermore, the real-world case studies found that the improved flexibility and performance in industries highlighted the importance of adaptive solutions in data-intensive applications.

Zhang et al. [19] mentioned that general query optimizer has bad performance when dealing with real customer workloads that are complex and diverse. The authors aim to build a QO-Advisor to address this issue. It improves query optimization by recommendation using contextual bandits and validation using supervised learning and real execution tests. The contextual bandit model will suggest small and safe rule changes to produce potentially better query plans and validation models will evaluate these suggestions using real execution metrics. Then, it will deploy rule changes that show clear and reliable improvement as optimizer hints for future queries. Results show that QO-Advisor reduces latency by 8.9%, reduces PNhours (resource usage) by 14.3% and reduces In vertices by 52.8%. The contextual bandit model finds 3 times more better plans and 2 times fewer worse plans compared to the random rule selection.

Rahman et al. [20] reviewed different advanced query optimization techniques to compare how different techniques perform in big data analytics scenarios. The study uses qualitative research approach to evaluate advanced SQL query optimization techniques. They are tested on large datasets to compare the execution time and resource usage. Results show that multi-level indexing reduces data retrieval time by 40% due to its hierarchical structure that provides precise paths and sped up query responses. Query rewriting reduces execution times by 35% particularly effective for multi-join and subquery scenarios. Dynamic query execution

plans are able to adjust execution strategies in real time based on current data states and workloads and improved resource efficiency by 25%. Database administrators also mentioned that these advanced techniques have minimal learning curve and are more reliable.

Pavlopoulou et al. [21] investigated the constant problem of suboptimal query execution in Big Data Management Systems. The study proposed a runtime dynamic optimization approach that executes all the predicates local to each dataset first and then uses the resulting intermediate output to re-optimize the remaining query by means of staged re-optimization points. This technique replaces uncertain estimates with real-intermediate statistics for more accurate join ordering and selection of plans. Their investigations highlight that although the approach incurs some overhead from materializing the intermediate results, the cost was far outweighed due to the performance gains from significantly improved execution plans. This demonstrates the effectiveness of dynamic runtime optimization for complex, large-scale queries.

Kalumin and Deshpande [22] mentioned that many-to-many joins in machine learning and graph analytics generate large intermediate results which made the query execution inefficient. Therefore, the authors developed a unified cost model to predict the number of probes for different execution strategies. They formulate the join order optimization problem, providing both exhaustive and heuristic algorithms, and implement a vectorized query engine to compare six variants in practice, leveraging SIMD-based batch processing, bitvector pruning, and semi-join full reduction to reduce intermediate results. After experimental evaluation, results shows that COM (factorized execution with avoidance of redundant probes) outperform traditional STD-based methods. COM's ability to avoid redundant probes is the main reason for this major improved performance.

5) Hardware-Accelerated Optimization (GPU-based)

Hardware-accelerated optimization leverages GPUs to speed up complex join operations. By using parallel processing or splitting the work across many cores simultaneously, these techniques reduce execution time and

handle large queries more efficiently than traditional CPU-based methods.

Mageirakos et al. [23] mentioned that query optimizer takes time to find an optimal plan for complex join query which slows down the performance. Existing join order optimization techniques are inefficient and do not parallelize well, so adding more CPU cores or GPUs doesn't significantly improve their speed. Therefore, the study proposed a GPU-accelerated optimizer, MPDP that speeds up join order optimization by combining two types of enumeration methods and exploiting massive GPU parallelism. It combines vertex-based and edge-based enumeration. For much larger queries, the authors proposed two simple heuristics, IDP2-MPDP and UnionDP, which help to break large queries into smaller parts so that it's easier to perform the join. After testing, it shows that MPDP is 3 times faster than DPSUB (GPU), 30 times faster than DPE-DPCCP, and more than 3 orders of magnitude faster than PostgreSQL plan. Heuristic solutions for very large queries also perform better compared with PostgreSQL and LinDP.

Rui et al. [24] stated existing methods assume that both input tables and intermediate join results can fit into a single GPU's limited memory. This causes issue when tables are large and result in excessive data transfer between CPU and GPU. The paper introduces three multi-join algorithms including nested-loop join to handle complex join conditions, Global sort-merge (GSM) join for quality or range joins, and hybrid join that combines radix partitioning and sort-merge to make equi-joins and integer-based range joins more efficient. Results shows that hybrid join has better performance compared to GSM and nested loop joins. GSM is heavily limited by data transfer bandwidth. Overall, the hybrid GPU join achieves up to 25× speedup over CPU-based algorithms and outperforms prior GPU join methods.

C. Comparison Table

Table 1 shows a summary and comparison of the techniques used in different studies.

Table 1: Comparison Table of Literature Review

Reference	Problem Statement	Algorithm/Methodology	Key Findings
AI-Driven / Machine Learning-Based Query Optimizers			
Sassi and Jaziri [6]	Traditional optimizers fail with complex joins under dynamic workloads.	Builds GRQO framework combining reinforcement learning (Proximal Policy Optimization) with graph-based schema representation. Learns optimal join orders dynamically. Updates policies using execution feedback and models table relationships as graphs for better cardinality estimation.	Achieved ~25% faster query execution and ~30% better scalability. Reduced resource overhead and improved adaptability for large-scale workloads.

Panwar [7]	Cost-based optimizers cannot handle dynamic workloads and complex joins.	Uses AI-driven query optimization with machine learning models trained on historical and real-time query data. Predicts efficient execution plans. Adapts to workload changes and retrains periodically using actual execution results.	Delivered significant speed improvements and better scalability. Reduced resource utilization compared to traditional optimizers.
Bao et al. [4]	Traditional ML-based query optimizers are slow, hard to retrain, and costly to integrate with real DBMS.	Generates multiple query plans using different hint sets. Uses Tree-Convolutional Neural Network (TCNN) to predict plan performance and Thompson Sampling to select plans. Retrains periodically using actual execution results and bootstrapping.	~50% improvement in execution time Reduces tail latency, improving overall speed. Rarely causes regressions and some queries improved by 20+ seconds.
Karri [8]	Traditional relational query optimizers struggle with unstable plans and unpredictable tail latencies.	AI-based optimizer with three modules: Learned Cardinality Estimator (LCE): Predicts row counts. Neural Cost Corrector (NCC): Corrects cost estimate errors. Reinforcement-Learning Planner (RLP): Suggests optimal join order.	~11% improvement in accuracy, ~9% in F1 score. ~40% reduction in execution time for complex/multi-join queries. CPU and memory usage reduced by 25–30%.
Messaoudi and Touahria [9]	Multi-join query optimizations are inefficient due to traditional optimizers generating poor plans because of huge search space and inaccurate cardinality estimates.	ML model encodes join plans as feature vectors where it trains linear and piecewise-linear models to predict plan cost and cardinalities. These models are then integrated into optimizer for pruning.	Improves plan selection accuracy, enhances stability, and maintains deterministic optimizer behavior.
Ji et al. [10]	Conventional optimization techniques find it difficult to efficiently explore the vast search space, resulting in suboptimal query plans	Dynamic double deep Q-network (DDQN) approach was used for joining order optimization (DDOS). Models join order as a Markov decision process. Dynamic progressive search was employed.	Outperforms dynamical programming, heuristics, and standard reinforcement learning techniques and reduced query time.
Karri and Muntala [11]	Traditional optimizers produce inaccurate cardinality estimates and unreliable plans.	Predict cost/cardinality, RL-based join/operator decisions, and bandit-based online adaptation.	Reduces execution time, improves reliability, handles cold start and privacy constraints.
Evolutionary, Metaheuristic, and Combinatorial Optimization			
Du et al. [12]	Distributed databases face CPU, I/O, and network latency challenges.	Implements multi-source mating selection evolutionary algorithm combining genetic algorithms, artificial bee colony strategies, and spectral clustering. Generates diverse execution plans. Accelerates convergence, and considers CPU, I/O, and network costs.	Reduced query response time and execution cost by a large margin. Improved convergence speed and maintained diversity in plan search.

Gaikwad et al. [13]	Exhaustive join enumeration is expensive for multi-join queries.	Apply combinatorial optimization techniques such as genetic algorithms, simulated annealing, and tabu search. Explores large search spaces efficiently. Avoid getting stuck in local optima and update solutions regularly based on cost feedback.	Improved global query performance significantly. Avoided local optima and reduced execution time for complex multi-join queries.
Karthikeyan et al. [14]	Multi-join optimization is difficult due to large plan search space and dynamic environments.	Q-Learning and Ant Colony Optimization (ACO) are used to explore join sequences and adapt plans.	Execution cost is reduced, join order selection is improved and adapts well in distributed environments.
Cardinality Estimation Improvements			
Justen and Boehm [15]	Multi-join cardinality estimates are inaccurate, particularly for complex join graphs.	OmniSketch which is a probabilistic structure combining count-min sketch and K-miniwise hashing for multi-join estimation.	Up to 1,077 times reduction in result size error. Up to 3.19 times faster execution. Strong performance on SSB-skew.
Han et al. [16]	Lack of real-world evaluation on whether ML cardinality estimators truly improve overall query plans.	Benchmark (STATS-CEB) integrated into PostgreSQL. Proposed P-Error metric forend-to-end evaluation.	Low Q-Error represent poor plan whereas P-Error better reflects true plan performance. Highlights trade-offs in ML estimators.
Adaptive & Runtime Optimization			
Zhang et al. [17]	RL-based optimizers are complex and costly; need simpler alternatives.	Developed adaptive query processing strategies in PostgreSQL dynamically switches between join algorithms (Nested Loop and Hash Join) based on runtime conditions. Uses Lookahead Information Passing (LIP) to prune unnecessary tuples early.	Outperformed RL-based optimizers in many cases. Reduced execution time without expensive training. Improved robustness and interpretability.
Rodriguez and Turner [18]	Traditional optimization fails for big data (Volume, Velocity, Variety).	Reviews distributed query execution, parallel processing, and machine learning-based optimization. Proposes hybrid frameworks combining cost-based and AI-driven methods for scalability and adaptability.	Hybrid frameworks achieved best results for big data workloads. Real-world case studies showed improved performance and scalability in finance and healthcare applications.
Zhang et al. [19]	Traditional query optimizers perform poorly on complex, real customer workloads. ML-based steering methods can cause regression due to inaccurate cost models.	QO-Advisor uses contextual bandits to recommend small, safe rule changes. Validation with supervised learning and real execution tests to evaluate suggestions. Deploys only rules that reliably improve performance.	Latency was reduced by 8.9%. PNhours (resource usage) reduced by 14.3%. Vertices reduced by 52.8%. Contextual bandit finds 3 times better plans and 2 times fewer worse plans than random selection.
Rahman et al. [20]	Traditional query optimization (basic indexing, heuristics) is inefficient for large-scale data.	Qualitative study evaluating advanced SQL optimization techniques. Techniques tested: multi-level indexing, query rewriting, dynamic query execution.	Multi-level indexing: 40% faster data retrieval. Query rewriting: 35% faster execution, especially for multi-joins/subqueries.

		Data collected via performance metrics and feedback from database administrators.	Dynamic execution plans: 25% improved resource efficiency.
Pavlopoulou et al. [21]	Big Data systems suffer from inaccurate intermediate cardinality estimates which leads to bad plans.	Runtime dynamic re-optimization using staged re-optimization points and real intermediate statistics.	Major performance gains despite some overhead. Significantly better join ordering and plan quality.
Kalumin and Deshpande [22]	Many-to-many joins in machine learning and graph analytics produce large intermediate results, slowing query execution.	Developed a unified cost model to predict probes for different strategies. Formulated join order optimization with exhaustive and heuristic algorithms. Implemented vectorized query engine with SIMD batch processing, bitvector pruning, and semi-join reduction. Compared six execution variants in practice.	COM (factorized execution avoiding redundant probes) outperforms traditional STD-based methods. Performance gains mainly due to avoiding redundant probes.
Hardware-Accelerated Optimization (GPU-based)			
Mageirakos et al. [23]	Query optimizers are slow for complex join queries. Existing join order techniques are inefficient and do not scale well with more CPU/GPU cores.	GPU-accelerated optimizer MPDP using vertex-based and edge-based enumeration for join order & heuristics (IDP2-MPDP and UnionDP) to split very large queries. Exploits massive GPU parallelism for speed.	MPDP: 3 times faster than DPSUB (GPU), 30 times faster than DPE-DPCCP, more than 3 orders of magnitude faster than PostgreSQL. Heuristic methods outperform PostgreSQL and LinDP on very large queries.
Rui et al. [24]	Existing GPU-based join algorithms struggle with large tables. Assumes all data fits in GPU memory, causing excessive CPU-GPU data transfer.	Proposed three multi-join GPU algorithms: Nested-loop join for complex joins. Global sort-merge (GSM) join for range/quality joins. Hybrid join (radix partitioning + sort-merge) for equi-joins and integer-based range joins.	Hybrid join performs best among the three. GSM limited by data transfer bandwidth. Hybrid GPU join achieves up to 25× speedup over CPU algorithms and outperforms prior GPU join methods.

D. Writing the Literature Review

1) Synthesize the Findings

The recent studies are mostly focused on AI-driven and machine learning-based query optimizers. It is because it is important to address the limitations of traditional cost-based approaches. These limitations are mostly in managing inconsistent workloads, complex multi-join queries, and inaccurate quantity prediction. The techniques such as reinforcement learning (GRQO [6], DDQN-based DDOS [10], bandit algorithms [4]) and learned models for quantity prediction [8] [9] have showed significant improvements in performance, flexibility, and efficiency. Although these approaches can perform better than traditional optimizer techniques, they often face challenges across different workloads such as high training costs, training requirements, and limited expansion. Besides that, the concept and mixed methods provide a simpler solution that do not require training. It allows users to explore a huge query efficiently.

For instance, the methods are such as genetic algorithms, simulated annealing, and ant colony optimization [12] [13] [14]. However, these methods are lack of predictive accuracy in AI-based models. The flexible process techniques [17] [21] provide a simple solution that can adjust implementation strategies rapidly without needing complex learning steps. Subsequently, GPU-base optimizers [23] [24] influence the similarity to accelerate join operations and improved the performance up to 30 times.

2) Identify Gaps and Opportunities

Despite these advancements, there are also several gaps remaining. Most AI-driven solutions are difficult for practitioners to trust and adopt. It is because most of them struggle with integration into existing DBMS and lack explainability. While metaheuristic and adaptive methods are practical, their scalability is limited for heterogeneous or cloud-native environments. Furthermore, cardinality estimation often fails to translate accuracy gains into better

end-to-end query performance although their improvements are promising. This issue highlights the importance of having a new evaluation metrics like P-Error [16]. Besides that, hardware-accelerated solutions have limited their application in large-scale distributed system. It is because of the memory constraints and data transfer bottlenecks. Therefore, the future research should explore more on hybrid frameworks that combine AI-driven learning, metaheuristic adaptability, and hardware acceleration. This will help to ensure that research results can improve interpretability, cross-database transfer learning, and real-world validation under diverse workloads.

E. Summary of Literature Review

The literature review showed that many studies have used different approaches to improve query optimization. The qualitative results obtained from the studies proven that those techniques can improve performance and they are more effective compared to traditional cost-based methods. Although the solutions provide advantages and benefits the solutions but there are several limitations that need to be addressed. These include high training costs, limited explainability and integration of AI-based models and scalability issues. Besides, many approaches rely on iterative, guess-based optimization strategies making them perform poorly at recognizing the hidden relationships in data and hard to adapt when there is changes during execution. Building on these gaps, this research aims to explore an optimization approach that moves beyond isolated estimation techniques by incorporating learning and adaptive mechanisms to improve interpretability and applicability across diverse workloads.

III. RESEARCH METHODOLOGY

A. Introduction to the Methodology

This section discusses the methodology used to analyse and assess query optimization for complex multi-join queries in relational database systems. This study not only relates to the RDBMS experiments but also extends it by considering a NoSQL database environment. More precisely, this experiment incorporates MongoDB to show the execution of multi-collection join queries in a document-oriented NoSQL database with native aggregators. With this extension, query execution behavior can be investigated in a wider context of data models without deviating from the relational optimization on which we are mainly focused. A more structured, systematic approach is required to evaluate the benefits of those other strategies for optimization. This is caused by the inability of traditional cost-based optimizers and heuristic methods to accurately handle hidden data correlations and large join spaces.

Traditional cost-based optimizers and heuristics guided methods often encounter difficulties when predicting complex dependencies between tables. Therefore, in this study, the Causal-Graph Guided Reinforcement Learning (CG-RL) method was considered for addressing these problems. This approach models join order selection as a sequential decision-making. Instead of replacing the existing

methods, this approach investigates whether the execution cost can be reduced, and optimization efficiency can be improved over time using RL based on causality. While CG-RL is evaluated in a relational database setting, this study also investigates NoSQL query processing as an additional experiment. To illustrate join execution behavior in a schema-flexible setting, this research utilizes the \$lookup aggregation operator in MongoDB to execute equivalent join workloads. This extension to NoSQL is not presented as a learning-based optimizer, but rather an embedding of relational optimization results in alternative database paradigms.

The common research design, experimental setup, evaluation criteria, and tools adopted in this study will be presented in this methodology section. This section also describes how performance data as obtained is monitored and analyzed, thus, to establish the reproducibility and validity of each result. This study aims to have better performance than existing relational optimization models and at the same time be able to study how queries execute on NoSQL databases.

B. Research Design

In this study, different query optimization methods were quantitatively compared on complex multi-join queries for relational database systems. Instead of developing a database system for production use. In this research, it focuses on the ability of different optimization methods to perform under control and limited simulation constancy.

The study being quantitative, the performance indicators are based on measurable ones. These are cumulative metrics such as the cumulative cost of performing executions and the sum of rewards. These measures provide an objective assessment to compare the effectiveness and efficiency of each approach.

The study is centered on the comparison evaluation approach, which tests the supervised learning models against our proposed CG-RL Optimizer. Our supervised learning models are the Decision Tree, Random Forest, and Logistic Regression. All the techniques are compared to the same setting simulated on a database, with shared datasets, cost, and causal structures. With this design, this study ensures that if any performance discrepancy is observed in the result, then it is a product of optimization to approach itself rather than differences between experimental configurations.

Parallel to the relational optimization experiments, the design of the study also includes a section for running NoSQL database environment as an experimental program. This NoSQL extension is used to study the execution of the same logical multi-join workloads over a document-oriented data model. The NoSQL system is not treated as an optimizer itself and does not support join order optimization or learning. Instead, it provides an execution environment to study how query performance changes under a different database paradigm.

In contrast to the relational experiments, where optimization effectiveness is quantified using cumulative execution cost and reward-based measurements, the NoSQL part considers execution-level performance measures, like the time it takes to execute queries. This disassociation helps to maintain consistency in evaluation methodology and avoid inappropriate comparisons between learning-based optimizers and non-learning database systems.

C. Steps Involved in the Methodology

1) Identification of Research Objectives

The main purpose of this comparative study is to evaluate and compare selected techniques with further study on performance in a NoSQL system to find out which technique is the best. By clearly setting this goal, the research ensures that all coming up steps are aligned to produce a meaningful and actionable insight.

2) Literature Review and Analysis

This step focuses on finding relevant techniques based on defined selection and standards instead of conducting a traditional literature review. These techniques are mostly selected from recent studies that are published in credible sources such as IEEE Xplore, ACM Digital Library, and ScienceDirect. The selection guidelines require that the techniques must be frequently cited, applicable to relational databases, and reviewed in peer-reviewed studies within the last five years. However, the techniques that are designed for non-relational systems or lack performance standards are excluded. This approach help to ensures that only relevant and comparable methods are included in the study.

3) Criteria for Comparison

For the comparative study, a specific evaluation metric is developed to ensure fairness and consistency. These criteria are such as learning efficiency, join selection quality and optimization quality. Learning efficiency can show that whether the agent improve by learning from past experience and demonstrate adaptability. Subsequently, the join selection quality compares the decision and accuracy at each step. In addition, the quality of solution reviews the efficiency of reducing query costs. Besides that, we also make extra observation on the execution time and query behaviour of NoSQL performance. However, the learning-based metrics are only applied to relational optimization models. By using these standards, it provides a comprehensive framework for reviewing the strengths and weaknesses of each technique.

4) Data Collection

The data collection step includes combining performance data under controlled situations using benchmark datasets and standardized workflows. Subsequently, a test environment is also created using a relational database system such as PostgreSQL or MySQL. Besides that, the benchmark datasets like Transaction Processing Performance Council – H (TPC-

H) or Transaction Processing Performance Council – Decision Support (TPC-DS) are used to ensure the fairness in every query tasks. Each of the technique is implemented and tested using the same hardware and software setups to ensure fairness. Furthermore, the performance measurements allow for direct comparison across different studies because they are recorded and can be adjusted when necessary. Therefore, this controlled approach ensured that the results are reliable and comparable. The benchmark used in both relational and document-based formats are the same. It is because same benchmark is needed to support implementation in relational and NoSQL system.

5) Experimental Setup

For the experiment, the simulated database setting was created using Python programming language on Google Colab. The Causal-Graph Guided Reinforcement Learning (CG-RL) Optimizer was developed using the PyTorch framework and the Deep Q-Network Model, whereas the baseline models Decision Trees, Random Forest and Logistic Regression were developed using the scikit-learn library. The simulation setup contains several tables of different sizes that are randomly generated and connected by causal relations. This simulation environment was run on Google Colab which is computationally resourceful enough to allow for smooth optimization execution. The MongoDB was selected to represent the NoSQL DBMS because it allows supports complex aggregation pipelines. The workload includes synthetic analysis and query over different datasets with caused relationships. The metrics collected to measure performance are such as query execution time, optimization convergence rate, and resource application.

6) Implementation of Solutions

In this paper, the NoSQL database was used only as a query execution environment for performance monitoring, and it played no role in implementing or training optimization algorithms. The proposed Causal-Graph Guided Reinforcement Learning (CG-RL) Optimizer is implemented using reinforcement learning within a Python-based simulated database environment. Reinforcement learning will iteratively observe query states, select join actions, and refine the model based on the feedback from execution outcomes. To compare its efficiency and effectiveness, baselines models were also deployed within the same environment using supervised learning-based baselines with static decision policies. These baseline models join orders based on fixed state representations without adaptive learning. All approaches were evaluated under identical data settings and cost assumptions to ensure a consistent and fair comparison.

The proposed CG-RL Optimizer is tested in a Python-based simulated database environment. The environment consists of multiple tables and assigns each of them with an random size representing its volume of data. A causal relationship between tables may be defined as some Directed Acyclic Graph (DAG) where edges represent a preferred join

order such that query cost decreases along an edge. The environment allows the agent to repeatedly join tables, while keeping track of which tables are remaining and the cumulative cost of the join sequence.

Each state is a binary vector that represents which tables have not yet joined (1 for remaining unjoined table, 0 for table already joined). An action represents choosing one of the remaining unjoined tables to join next. This representation enables the agent to understand the current query execution state and make informed decisions about which join to perform.

The agent receives a reward according to how "expensive" it is to join a table: If a join follows the causal DAG, it's cheaper and rewards the agent with higher rewards. Joins that do not follow causal relationships are more expensive and therefore yield lower rewards. In practice, the reward equals negative join cost, meaning reward is higher when cost is low. It is adjusted based on whether the join respects causal order. In this way, the agent learns gradually to select join sequences that keep the overall query cost as low as possible.

The CG-RL Optimizer employs a Deep Q-Network (DQN) based reinforcement learning agent that can learn an efficient sequence of table joins, which minimizes the overall query execution cost. The DQN models this join optimization problem as a sequential decision-making process where each action represents the selection of the next table to join.

The agent observes the state of the current query at every step, represented as a binary vector of which tables are unjoined. It then selects an action that involves choosing one table to join next based on the current state it observes. Thereafter, the action executes in the environment and receives the reward based on the cost of performing the join, and whether the selected join follows the pattern of causal relationships defined in the DAG.

This interaction is presented in the code shown in Figure 1.

```
def train_dqn(env, episodes=1000, gamma=0.95, epsilon_start=1.0, epsilon_end=0.05, epsilon_decay=0.995):
    state_dim = len(env.tables)
    action_dim = len(env.tables)
    model = DQNAgent(state_dim, action_dim)
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    loss_fn = nn.MSELoss()
    memory = ReplayMemory()
    epsilon = epsilon_start
    all_rewards = []
    for ep in range(episodes):
        state = env.reset()
        done = False
        total_reward = 0
        while not done:
            available_actions = [i for i, t in enumerate(env.remaining_tables)]
            state_tensor = torch.FloatTensor(state).unsqueeze(0)
            if random.random() < epsilon:
                action = random.choice(available_actions)
            else:
                with torch.no_grad():
                    q_values = model(state_tensor).numpy()[0]
                    full_q = np.full(action_dim, -np.inf)
                    full_q[available_actions] = q_values[available_actions]
                    action = np.argmax(full_q)
            next_state, reward, done, table = env.step(action)
            total_reward += reward
            memory.push(state, action, reward, next_state, done)
            state = next_state
```

Figure 1: DQN table join process

- state represents the current query state
- action is the chosen table to join
- reward corresponds to the negative join cost adjusted by causal relationships
- next_state represents the updated state after the join.

Q-Network Architecture and Action Evaluation

The DQN takes the current state vector as input and produce a Q-value as output for each possible table. This Q-value represents the expected cumulative reward of selecting that table as the next join. The higher Q-value indicates the better and suitable the join decision. This interaction is presented in the code shown in Figure 2.

```
dqn(env, episodes=1500, gamma=0.95, lr=0.001, batch_size=64, target_update=50):
    dim = len(env.tables)
    _dim = len(env.tables)
    = DQNAgent(state_dim, action_dim)
    _model = DQNAgent(state_dim, action_dim)
    _model.load_state_dict(model.state_dict())
    zer = optim.Adam(model.parameters(), lr=lr)
    n = nn.MSELoss()
    = ReplayMemory()
    n = 1.0
    s_per_episode = []

    in range(episodes):
        ate = env.reset()
        ne = False
        tal_reward = 0
        ile not done:
            available = list(range(len(env.remaining_tables)))
            if random.random() < epsilon:
                action = random.choice(available)
            else:
                q = model(torch.FloatTensor(state).unsqueeze(0))[0].detach().numpy()
                action = available[np.argmax([q[i] for i in available])]
            next_state, reward, done, _ = env.step(action)
```

Figure 2: Q-value for table selection

The agent chooses between actions using the ϵ -greedy strategy to balance between exploitation and exploration. This interaction is presented in the code shown in Figure 3.

```
def train_dqn(env, episodes=1000, gamma=0.95, epsilon_start=1.0, epsilon_end=0.05, epsilon_decay=0.995):
    state_dim = len(env.tables)
    action_dim = len(env.tables)
    model = DQNAgent(state_dim, action_dim)
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    loss_fn = nn.MSELoss()
    memory = ReplayMemory()
    epsilon = epsilon_start
    all_rewards = []
    for ep in range(episodes):
        state = env.reset()
        done = False
        total_reward = 0
        while not done:
            available_actions = [i for i, t in enumerate(env.remaining_tables)]
            state_tensor = torch.FloatTensor(state).unsqueeze(0)
            if random.random() < epsilon:
                action = random.choice(available_actions)
            else:
                with torch.no_grad():
                    q_values = model(state_tensor).numpy()[0]
                    full_q = np.full(action_dim, -np.inf)
                    full_q[available_actions] = q_values[available_actions]
                    action = np.argmax(full_q)
            next_state, reward, done, table = env.step(action)
            total_reward += reward
```

Figure 3: ϵ -greedy action selection

Most of the time, the agent selects the action with the highest predicted Q-value (exploitation). Sometimes, it may select a random valid action (exploration) as well in order to find possibly better join sequences and avoid being stuck in local optima.

Q-Value Update and Learning Process

The DQN does so by updating its Q-values with the aid of the Bellman equation (1), which considers the immediate reward and expected future rewards:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a') \quad (1)$$

In implementation, Double DQN with a target network is used to enhance stability in training. An online network selects the best next action, while the target network estimates

its Q-value. This interaction is presented in the code shown in Figure 4.

```
if len(memory) >= batch_size:
    s, a, r, s2, d = memory.sample(batch_size)
    q_curr = model(s).gather(1, a)
    # Double DQN update
    next_actions = model(s2).argmax(1).unsqueeze(1)
    q_next = target_model(s2).gather(1, next_actions).detach()
    q_target = r + gamma * q_next * (1 - d)
    loss = loss_fn(q_curr, q_target)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Figure 4: Double DQN: Q-value prediction with target network

The loss between predicted and target Q-values is computed using Mean Squared Error (MSE), and the network parameters are updated via backpropagation. This interaction is presented in the code shown in Figure 5.

```
if len(memory) >= 32:
    s_batch, a_batch, r_batch, s_next_batch, d_batch = memory.sample(32)
    s_tensor = torch.FloatTensor(s_batch)
    a_tensor = torch.LongTensor(a_batch).unsqueeze(1)
    r_tensor = torch.FloatTensor(r_batch).unsqueeze(1)
    s_next_tensor = torch.FloatTensor(s_next_batch)
    d_tensor = torch.FloatTensor(d_batch).unsqueeze(1)
    q_current = model(s_tensor).gather(1, a_tensor)
    q_next = model(s_next_tensor).max(1)[0].unsqueeze(1).detach()
    q_target = r_tensor + gamma * q_next * (1 - d_tensor)
    loss = loss_fn(q_current, q_target)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

epsilon = max(epsilon_end, epsilon * epsilon_decay)
all_rewards.append(total_reward)
```

Figure 5: Q-value Loss Calculation and Network Update

Experience Replay and Training Stability

The agent will store transitions in a replay memory, which improves learning stability and reduces correlation between consecutive experiences. This interaction is presented in the code shown in Figure 6.

```
for ep in range(epochs):
    state = env.reset()
    done = False
    total_reward = 0
    while not done:
        available_actions = [i for i, t in enumerate(env.remaining_tables)]
        state_tensor = torch.FloatTensor(state).unsqueeze(0)
        if random.random() < epsilon:
            action = random.choice(available_actions)
        else:
            with torch.no_grad():
                q_values = model(state_tensor).numpy()[0]
                full_q = np.full(action_dim, -np.inf)
                full_q[available_actions] = q_values[available_actions]
                action = np.argmax(full_q)
        next_state, reward, done, table = env.step(action)
        total_reward += reward
        memory.push(state, action, reward, next_state, done)
        state = next_state
```

Figure 6: Experience Replay Memory for Stable Learning

During training, the agent samples mini-batches of 64 experiences uniformly at random. This interaction is presented in the code shown in Figure 7

```
if len(memory) >= batch_size:
    s, a, r, s2, d = memory.sample(batch_size)
    q_curr = model(s).gather(1, a)
    # Double DQN update
    next_actions = model(s2).argmax(1).unsqueeze(1)
    q_next = target_model(s2).gather(1, next_actions).detach()
    q_target = r + gamma * q_next * (1 - d)
    loss = loss_fn(q_curr, q_target)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Figure 7: Mini-Batch Sampling from Replay Memory

This allows the agent to learn from past experiences and generalize across different join configurations.

A target network is periodically updated to further stabilize training. This interaction is presented in the code shown in Figure 8.

```
if ep % target_update == 0:
    target_model.load_state_dict(model.state_dict())

return model, rewards_per_episode
```

Figure 8: Periodic Target Network Update

An episode represents a complete join sequence, starting with all tables unjoined and ending when every table has been included in the join order. Over many episodes, the agent learns to assign higher Q-values to join actions that lead to lower overall query costs and respect causal dependencies. Through this learning process, the CG-RL Optimizer is able to predict efficient join orders for complex multi-join queries, which helps in improving adaptability compared to static baseline models.

For comparison, conventional machine learning models including Decision Tree, Random Forest and Logistic Regression are trained on the same state-action dataset. Unlike the DQN agent, these models do not adaptively learn sequences and make decisions based on static patterns. All models are evaluated using identical data settings to ensure a fair comparison.

7) Evaluation and Comparison

The existing solutions or baseline models are compared with the proposed solution through multiple quantitative criteria. To assess the optimization quality, the proposed CG-RL Optimizer is evaluated using cumulative reward and average episode reward. Reward has a negative relationship with join cost, the higher cumulative reward, the lower total query cost. Moving on, reward progression over episodes and learning curve plot demonstrates the learning efficiency of CG-RL Optimizer. As for the join selection quality, accuracy, precision, recall and F1-score are used to evaluate how often the model pick the right tables to join at each step.

8) Validation of Results

The results were validated for reliability and reproducibility. For the CG-RL Optimizer, separate test episodes were used to evaluate the learned join strategies to make sure its performance was assessed on data not seen during training. Learning progression was monitored through cumulative reward trends to verify continuous improvement.

Baseline models were also evaluated on separate datasets generated from the same environment to ensure a fair comparison.

D. Tools and Technologies Used

This study utilized a mix of frameworks for programming, machine learning libraries, tools for simulation to implement and evaluate the chosen query optimization techniques. Python was used on the Google Colab platform for all the development, experimentation, and evaluation. This is due to this platform offers a cloud-based environment suitable for quick prototyping a computational experimentation.

The Causal-Graph Guided Reinforcement Learning (CG-RL) Optimizer that was proposed was built using PyTorch. PyTorch was used to build and train a neural network-based value learning agent. Its features helped to make the model training easier with its support for automatic differentiation, optimization routines and neural network abstractions. This helped implement value-based reinforcement learning with target networks and experiment replays. Epsilon greedy exploration strategy was followed for the learning process to balance exploration and exploitation during training.

Network X library was utilized to create a directed causal graph that represented the true join relationships. This is to model causal relationships between database tables and simulate hidden data dependencies. NumPy was used to carry out data processing and numerical operations whereas Matplotlib was used to display the learning curve and training performance trends. These tools together supported the implementation, evaluation, and interpretation of the proposed optimization approach.

MongoDB was used as the NoSQL DBMS to support the NoSQL-based tests through the MongoDB Atlas platform. This tool is chosen to use in this study because it allows flexible models and a visualization of separated data. The database supported the performance of aggregation pipelines that similar to join-like operations in No-SQL workloads. MongoDB was used to check the NoSQL comparisons of equivalent workloads execution behaviour in handling non-relational query execution strategies.

E. Flow Diagram or Architecture

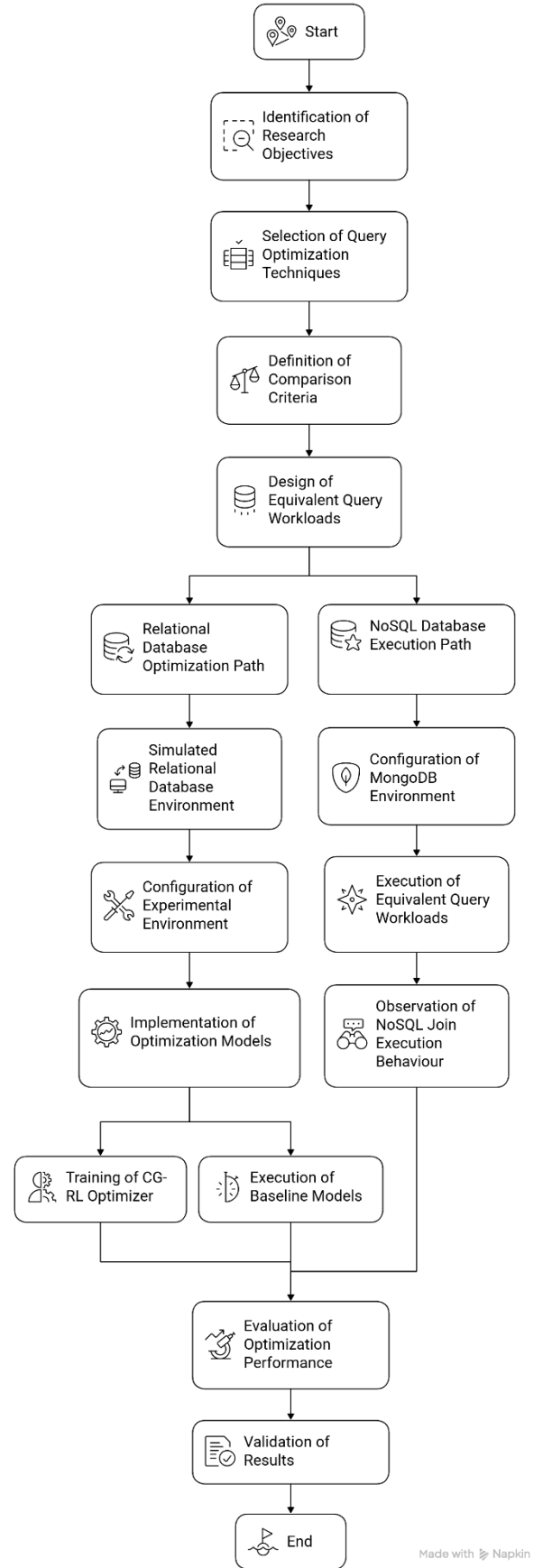


Figure 9: Flow Chart for Methodology

Figure 9 above illustrates the overall experimental procedure used in the present study. This figure was generated using the Napkin AI platform. For all experiments, both datasets and complex multi-join query workloads are generated for reasons of consistency. Then, a simulated relational database environment was set up in Python including the definition of join cost models as well as causal graphs that represent table dependencies.

Causal-Graph Guided Reinforcement Learning (CG-RL) Optimizer was implemented, which is trained in this setting to learn optimal join orders concurrently with learnt baselines based on supervised learning. The join sequence that produces the best execution plan is finally validated using the cumulative execution cost and reward-oriented performance metrics.

The equivalent logical query workloads are deployed in a NoSQL data base environment, which is the MongoDB, using a document-based data model. The NoSQL system is only employed to execute queries and monitor the performance, which means that it has no logic of its own optimization. Finally, the findings obtained from the relational optimization experiments and NoSQL execution are examined and validated to understand performance behaviour in other database paradigms

F. Challenges and Limitations

Throughout the research process, there are multiple challenges and limitations were faced. First and foremost, there is one major challenge which is the reliance on performance data reported in existing literature. It might not be accurate because the literatures did not conducted experiments on real-world production databases. However, this approach is flexible because different studies have used it in different datasets, hardware systems, and workflow patterns. It is important because it allows us to make direct comparisons with higher accuracy.

Besides that, the study did not include a real-time review due to limited resources. The analysis was based on fixed standards and reported results that may not fully measure performance under a complex situation. Subsequently, a real-world integration issue is also a big challenge for us. It is because many advanced techniques require long training time, specialized hardware, and complex integration with existing DBMS. Furthermore, mongoDB does not support explicit query plans or cost-based join optimization, like relational systems, making comparison of join order strategies difficult. Accordingly, the evaluation of MongoDB was limited to observation of execution behaviour rather than a comparison at the level of optimizers. Lastly, the differences in analysis tools in different studies also brought us challenges in ensuring consistency.

G. Summary

This section explained the methodology that is used in this study to analyze and compare existing techniques. The research has used a comparative study design that highlights academic review. It also provided a standard for us to find

previous studies. The key steps in the methodology are such as identifying selection standards, relevant techniques, and performance metrics. These steps help us to combine findings from different sources with a structured and transparent process.

Although there are some limitations in this methodology, it still provides a complete framework for us to make decisions in selecting efficiency techniques. By reviewing the existing techniques carefully, this solution supports the research objective which is comparing the performance and efficiency of different techniques. Lastly, the methodology also works as an important connection between the literature review and the results. It is because it provides a meaningful insight for us to have more understanding on the current optimization strategies' strengths and weaknesses.

IV. RESULTS AND DISCUSSION

A. Introduction to the Results and Discussion Section

This section demonstrates the performance comparison of different query optimizer strategies, including the proposed Causal-Guided Reinforcement Learning Deep Q-Network (CG-RL DQN). The purpose is to investigate and explain the behaviour of these techniques for complex multi-join queries over relational database management system (RDBMS). The performance measurements will be quantified by several parameters such as cumulative rewards, learn efficiency and join select quality like, accuracy, precision, recall and F1-score, which given an insight into optimization effect of the methods and accuracy level. The proposed CG-RL is compared directly with the baseline models to highlight its strenghts and weaknesses.

B. Presenting Results

1) Organization of Results

The results are organized according to the main objective of this study which is to assess how well a Causal-Graph Guided Reinforcement Learning approach can lower join execution costs and improve optimization efficiency over time. The analysis begins with optimization quality metrics and learning behaviour. It is then followed by comparisons of join selection quality against baseline models.

2) Quantitative Results

Table 2 summarizes the join selection quality of the CG-RL DQN agent and the supervised baseline models using Accuracy, Precision, Recall, and F1-Score.

Table 2: Join Selection Performance Comparison Using Classification Metrics

Model	Accuracy	Precision	Recall	F1-Score
CG-RL DQN	0.6000	0.6000	0.4333	0.4833
Decision Tree	0.6740	0.6998	0.6383	0.6616
Random Forest	0.6660	0.6876	0.6354	0.6564

Logistic Regression	0.6670	0.6816	0.6216	0.6445
---------------------	--------	--------	--------	--------

Although the supervised models had better classification-based metrics, the CG-RL DQN agent shows competitive performance. Particularly, given the complexity of sequential decision-making and delayed reward optimization.

```

=== Optimization Quality ===
DQN Cumulative Reward      : -2956.94
Optimal Cumulative Reward  : -1.07
Average Episode Reward     : -1.97

```

Figure 10: CG-RL Optimization Quality Results

```

=== Learning Efficiency Summary ===
Initial Reward (Ep 1)      : -3.65
Final Reward (Ep 1500)    : -0.51
Best Episode Reward       : -0.23

```

Figure 11: CG-RL Learning Efficiency During Training

As shown in Figure 10 and Figure 11, the optimization behaviour of the CG-RL DQN agent was evaluated using cumulative reward and learning progression metrics. A cumulative reward of -2956.94 was achieved by the DQN agent compared to the optimal policy cumulative reward of -1.07 . This emphasizes that there are some convergence toward cost-efficient join sequences. Additionally, the value of the average episode reward increased from -3.65 in the early training to -0.51 in the final episode, which shows a clear learning progress.

3) Graphical Representation of Results

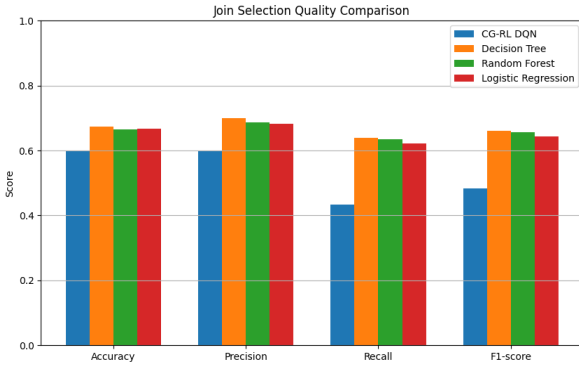


Figure 12: Join Selection Quality Bar Chart

Figure 12 shows the bar chart of the join selection quality of CG-RL DQN against the baseline models which are Decision Tree, Random Forest, and Logistic Regression. The metrics included are accuracy, precision, Recall, and F1-Score.

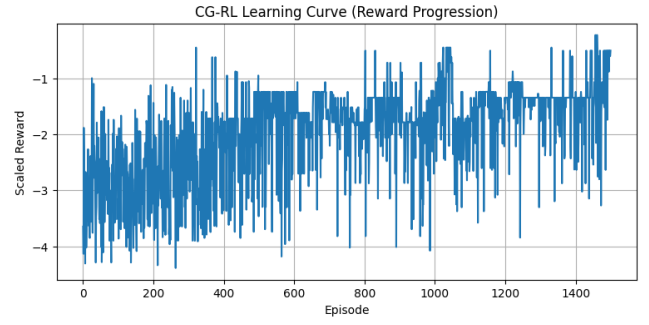


Figure 13: Learning Curve of the CG-RL Agent Across Training Episodes

Figure 13 shows the learning curve of the CG-RL agent. It shows steady improvement in reward values across training episodes.

4) NoSQL Query Observation Results (MongoDB)

This section observes the results from executing equivalent multi-join workloads in MongoDB. Note that accuracy, precision, recall, F1-score and reward are only relevant for relational optimization models and do not apply to the execution of MongoDB queries.

To compare equivalent relational optimization experiments, the same logical join workloads were performed in MongoDB using aggregation pipelines with \$lookup. As cost-based join reordering is not possible with MongoDB and learned optimization when reusing join orderings from previous queries was used, no training metrics such as reward or accuracy were considered. The execution pattern, however, was noted only qualitatively in terms of query completion and execution plausibility. The results show MongoDB can evaluate multi-collection join workloads but in a predefined order of the pipeline with no adaptive optimization. This further strengthens the need for learning-based optimizations in relational systems and also reveals inherent differences in execution models between relational and document databases.

C. Comparative Analysis and Discussion

1) Evaluation Metrics Considered

a) Join Selection Quality

Accuracy metric was used because it measures the overall correctness of the model's predictions. The CG-RL achieved the accuracy value of 0.6000 which is lower compared to the supervised baseline models where they achieved a value of 0.6740, 0.6660, and 0.6670 for Decision Tree, Random Forest and Logistic Regression respectively. This indicates that while the reinforcement agent learns join strategies that are aware of the costs, this agent does not prioritize exact imitation of the oracle join labels as effectively compared to the supervised classifiers that are directly trained to optimize prediction accuracy.

This precision metric was considered as it reflects the model's ability to avoid incorrect join selections. The CG-RL got the precision value of 0.6000, which is lower compared to the rest of the baseline models. The baseline models achieved a score of 0.6998, 0.6876, and 0.6816 for Decision

Tree, Random Forest and Logistic Regression respectively. These results suggest that supervised learning models are more consistent when it comes to replicating oracle join options. Whereas the reinforcement learning agent explores alternative join orders that could differ from the oracle labels. However, they could still be beneficial from a cost optimization perspective.

Recall metric measures how well the model can identify all relevant optimal join actions. A recall value of 0.4333 was obtained by the CG-RL whereas Decision Tree scored a value of 0.6383, Random Forest scored 0.6354 and Logistic Regression scored a value of 0.6216. This indicates that the reinforcement learning agent has exploration-driven nature. This is because the agent prioritizes discovering lower cost execution ways instead of exhaustively reproducing oracle join decisions.

F1-score balances the precision and recall and provide a holistic view of model performance. A value of 0.4833 was obtained by CG-RL whereas Decision Tree scored 0.6616, Random Forest scored 0.6564, and Logistic Regression scored 0.6445. The baseline models outperformed the CG-RL DQN agent when evaluated purely on label-based join selection accuracy.

However, this result obtained does not undermine the overall effectiveness of the proposed CG-RL approach. The reinforcement learning model focuses on reducing cost of execution rather than perfectly imitating an oracle. Hence, lower classification-based metrics can be expected. At the same time, the agent shows better performance in query execution efficiency.

The supervised baseline models perform better compared to CG-RL in classification-based metrics. This reflects their effectiveness in learning fixed input-output mappings when optimal actions are clearly labelled. However, CG-RL DQN agent adds complexity to its learning as it infers optimal behaviour through interaction with the environment and delayed reward signals.

b) Optimization Efficiency and Learning Behaviour

The CG-RL consistently reduces execution costs over time despite having lower classification scores. From -3.65 in the initial episode to -0.51 in the final episode, this result shows an improvement. It also shows that the agent learns cost-efficient join ordering strategies through sequential decision-making. The static supervised models do not capture this learning behaviour.

2) Justification of Results

The observed performance patterns result from the fundamental differences between reinforcement learning and supervised learning. While the supervised models treat join selection as a separate classification task, the CG-RL methods explicitly model the long-term effect of each join decision on overall execution costs. The agent is also guided by the causal relationships in the reward structure thus making the join sequences more efficient.

3) Strengths and Weaknesses

The key strength of the CG-RL approach is it has a great ability to model sequential dependencies and learn optimization strategies through environmental interaction. However, its weakness includes that this approach requires more computational resources during training and does not always outperform the supervised models especially in classification metrics. On the other hand, supervised models can train faster and have higher accuracy. However, these models are not able to optimize long-term execution costs in dynamic join ordering scenarios.

D. Overall Discussion

1) Summarize Findings

The results show that the proposed Causal-Graph Guided Reinforcement Learning (CG-RL) Optimizer did not performed better than the traditional supervised learning models across all evaluation metrics. CG-RL had achieved an accuracy of 0.60, precision of 0.60, recall of 0.43, and an F1-score of 0.48. It is slightly lower comparing to other models such as Decision Tree, Random Forest, and Logistic Regression. However, the reinforcement learning agent showed a strong learning potential over time. These results show that CR-GL provides a high adaptability and long-term improvement potential while a supervised model can perform better in accuracy.

2) Implications of Findings:

The findings suggest that reinforcement learning-based optimizers like CR-GL are encouraged to use when query patterns change frequently and long-term cost reductions are limited. Although other supervised models are performing better in accuracy, they are lack of adaptability and cannot improve like CR-GL in the future. Therefore, organisations that handle a large-scale analytical workload should implement CR-GL. It is because it can deliver a better performance than other models in a long-term situation. However, it is only suitable for systems with sufficient resources and acceptance for initial inefficiency due to the training costs and hardware requirements.

3) Address Limitations

One of the limitations of the CR-GL is it had a lower accuracy than other supervised models in short-term situations. The proposed optimizer requires a significant amount of training time and hardware requirements. This highlights that reinforcement learning needs more training before achieving a higher accuracy. Therefore, It will become a big challenge for organizations that have limited infrastructure. Subsequently, the generated datasets and controlled situations may not fully represent the quantity and complexity of real-world systems. Hence, the results may be different for non-relational or distributed systems.

E. Critical Evaluation

The primary objective of this study is to analyse and evaluate the existing query optimization techniques for complex multi-join queries and to present a new approach based on causal relationship-driven reinforcement learning.

Analysis of the results shows that the CG-RL is able to learn join orders that optimize the cost of query processing across episodes because there is an upward trend in cumulative rewards and improvement in average episode reward. These results demonstrate the effectiveness of the learning agent in adapting to and learning from complex query processing scenarios.

However, there are some limitations in the CG-RL approach. In relation to the quality of the join selection task, which is evaluated based on accuracy, precision, recall, and F1-Score, CG-RL performs poorly compared to supervised learning models like Decision Tree, Random Forest and Logistics Regression. This shows that the agent learns sub-optimal strategies for performing joins especially during the early episodes due to the exploration component in the reinforcement learning. The training process for this approach seems computationally expensive and might be a concern for practical scenarios.

Nonetheless, this solution has a unique advantage too including its ability to adaptively improve join strategies over time and consider long-term cumulative costs rather than just immediate predictions. The findings therefore highlight a trade-off between adaptive learning with potential cost reduction and the lower short-term accuracy as well as high computational demands of the approach.

F. Linking to Literature Review

The findings for this study align mostly with other recent works on AI-powered and reinforcement learning-based query optimizers. The same as what is observed in other studies like Sassi and Jaziri [6] and Ji et al. [10], the CG-RL demonstrates the effectiveness in the learning process of join orders that can reduce the overall query cost. Similar to the approaches proposed by Panwar [7] and Karri [8], CG-RL is effective in improving of long-term execution efficiency by considering cumulative rewards.

However, the results also pointed out the limitations that are identified in some previous works. For instance, the challenge of high training overhead and poor convergence speed in the reinforcement learning based optimizers are clarified in the studies by Bao et al. [4] and Karri & Muntala [11]. From the results, the accuracy, precision, recall and F1 score of the proposed CG-RL are poor when compared to the supervised learning approaches. This confirms that reinforcement learning approach offers adaptive learning but lacks the decision accuracy in the initial suboptimal joins.

In short, the results shows that AI-based solutions and reinforcement learning approach can improve the process of query optimization, at the same time, they also highlight the trade-offs as discussed in existing literature, such as training cost, accuracy of decision-making in the early stage and convergence time.

G. Conclude the Results and Discussion

In conclusion, the quantitative results obtained shows that the accuracy of the supervised learning models is clearly

higher when selecting joins. However, the decision rules generated are static and not adaptive. On the other hand, the learning abilities of the proposed model, CG-RL, are proven by the increase in cumulative rewards and the upward trend learning curve, indicating its effectiveness in reducing overall query cost over time. The drawbacks for the adaptive learning is the lower short-term decision accuracy and higher training complexity. Overall, the findings shows that causal-guided reinforcement learning is a feasible approach for optimizing complex multi-join queries, particularly in dynamic environments, while highlighting the need to balance adaptability with efficiency and stability in practical query optimization systems. All codes, datasets and simulation materials used in this project will be shared in a public GitHub repository so others can reproduce the results:

<https://github.com/Keertana0307/CGRL-Query-Optimization/>

V. CONCLUSION AND FUTURE WORK

A. Summary of Key Findings

This study investigated the use of a Causal-Graph Guided Reinforcement Learning to optimize join orders in relational database systems. The results from the experiments demonstrate that the proposed approach, the CG-RL DQN agent, is able to learn cost-effective join strategies over time. This was proven by steady improvements in overall and per-episode rewards.

B. Link Back to Objectives

The primary goal of this study was to examine and evaluate existing optimization techniques for complex multi-join queries. This research also aimed to identify the potential for a novel reinforcement learning-based strategy approach that is causal-model guided. The purpose of this work was facilitated by developing Causal-Graph Guided Reinforcement Learning (CG-RL) Optimizer. The method was compared to traditional supervised learning methods in this study. The comparison emphasized on query execution time, learning behaviour and join selection quality. The conclusion that CG-RL does not always outperform all baseline models in classification models can be proven according to the result of the experiment. However, it can learn cost-effective join strategies over time and minimize execution leveraging the sequential decision-making. Furthermore, it was shown that important trade-offs, constraints, and practical aspects need to be weighted against between learning-based and conventional optimization approaches. This is important since it can contribute to an understanding of the factors that affect the effectiveness of a wide range of query optimization techniques.

C. Contributions of the Research

This study addresses this gap by proposing a systematic experimental framework to evaluate the reinforcement learning-based query optimizers. This work emphasizes the significance of information of causal in reinforcement learning agents and shows evidence for reducing the costs of learning-based join order optimization.

D. Implications for Practice

The results indicate that learning methods based on reinforcement learning, such as CG-RL, can be advantageous in dynamic database environments. This is because the patterns of queries and relationships between data become different over time in such environments. This class of approach could support existing optimizers by adapting to new workloads, exposing optimization opportunities that static models will overlook.

E. Limitations

The most significant limitation of this study is that it performed under idealized conditions in a simulated database environment where the data were generated synthetically. Due to this, artificial data may not be sophisticated enough to reflect the complexities of a real-world database. Another drawback is that in practice training reinforcement learning models tend to be much more computationally expensive than traditional supervised techniques.

F. Recommendations for Future Research

Further work is suggested to validate the proposed algorithm on real database workloads and extend the model by considering more sophisticated query operators. It is possible to explore the hybrid optimization approach that combines reinforcement learning techniques with classical cost-based or supervised methods.

G. Concluding Remarks

To summarize, this study demonstrates the advantage of using a causal-guided RL method for query optimization. Although the CG-RL method may not compare favorably with the existing learning-based methods, it provides a good degree of insights into learning-based optimization. Additionally, this approach serves as an intermediary point for future development in intelligent database management systems.

REFERENCES

- [1] S. Abu Salim, M. Saringat and S. Mostafa, "A Comparative Study of Data Management Systems," *Journal of Soft Computing and Data Mining*, vol. 1, no. 1, pp. 10-16, 2020.
- [2] P. K. Sadineni, "Comparative Study on Query Processing and Indexing Techniques in Big Data," in *2020 3rd International Conference on Intelligent Sustainable Systems (ICISS)*, Thoothukudi, India, 2020.
- [3] L. Hai, Z. Bao and Y. Peng, "A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration," *Data Science and Engineering*, vol. 6, pp. 86-101, 2021.
- [4] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh and T. Kraska, "Bao: Making Learned Query Optimization Practical," in *SIGMOD '21: Proceedings of the 2021 International Conference on Management of Data*, China, 2021.
- [5] R. Mancini, S. Karthik, B. Chandra, V. Mageirakos and A. Ailamaki, "Efficient Massively Parallel Join Optimization for Large Queries," in *Proceedings of the 2022 International Conference on Management of Data*, 2022.
- [6] N. Sassi and W. Jaziri, "Efficient AI-Driven Query Optimization in Large-Scale Databases: A Reinforcement Learning and Graph-Based Approach," *Mathematics*, vol. 13, no. 11, p. 1700, 2025.
- [7] V. Panwar, "AI-Driven Query Optimization: Revolutionizing Database Performance and Efficiency," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 72, no. 3, p. 18–26, 2024.
- [8] N. Karri, "AI-Powered Query Optimization," *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 2, no. 1, 2021.
- [9] J. Messaoudi and N. Touahria, "A Machine Learning Approach to Multi-Join Query Optimization in Large-Scale Relational Databases," *Orient Journal of Emerging Paradigms in Artificial Intelligence and Autonomous Systems*, vol. 15, no. 11, p. 1–15, 2025.
- [10] L. Ji, R. Zhao, Y. Dang, J. Liu and H. Zhang, "Query Join Order Optimization Method Based on Dynamic Double Deep Q-Network," *Electronics*, vol. 12, no. 6, p. 1504, 2023.
- [11] N. Karri and P. S. R. Pedda Muntala, "Query Optimization Using Machine Learning," *IJETSIT [Internet]*, vol. 4, no. 4, pp. 109-117, 2023.
- [12] Y. Du, Z. Ding, Z. Cai and Y. Chi, "Join Query Optimization in Distributed Database Based on Multi-Source Mating Selection Evolutionary Algorithm," *Cluster Computing*, vol. 28, no. 5, p. 281, 2025.
- [13] P. & D. S. Gaikwad, "Combinatorial Approaches to Database Query Optimization: Enhancing Efficiency and Performance," *International Research Journal on Advanced Engineering and Management (IRJAEM)*, vol. 3, no. 1, 2025.
- [14] M. P. Karthikeyan, K. Krishnaveni and D.-N. Le, "Analysis of Multi-Join Query Optimization Using ACO and Q-Learning," *International Journal of Computing and Digital Systems*, vol. 16, no. 1, pp. 1523-1533, 2024.
- [15] D. Justen and M. Boehm, "Join Cardinality Estimation with OmniSketches," 25 August 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2508.17931>. [Accessed 6 12 2025].
- [16] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cong, Y. Qin, A. Pfadler, Z. Qian, J. Zhou, J. Li and B. Cui, "Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation," 13 September 2021. [Online]. Available: <https://doi.org/10.48550/arXiv.2109.05877>. [Accessed 6 12 2025].
- [17] Y. Zhang, Y. Chronis, J. M. Patel and T. Rekatsinas, "Simple Adaptive Query Processing vs. Learned Query Optimizers: Observations and Analysis," *VLDB Journal*, vol. 34, no. 5, p. 62, 2025.
- [18] S. Rodriguez and M. Turner, "Optimizing Query Processing for Big Data: A Comprehensive Review," *European-American Journals*, 2023.
- [19] W. Zhang, M. Interlandi, P. Mineiro, S. Qiao, N. Ghazanfari, K. Lie, M. Friedman, R. Hosn, H. Patel and A. Jindal, "Deploying a Steered Query Optimizer in Production at Microsoft," in *SIGMOD '22: Proceedings of the 2022 International Conference on Management of Data*, Philadelphia PA USA, 2022.
- [20] M. M. Rahman, S. Islam, M. Kamruzzaman and Z. H. Joy, "ADVANCED QUERY OPTIMIZATION IN SQL DATABASES," *Academic Journal on Business Administration, Innovation & Sustainability*, vol. 4, no. 3, pp. 1-14, 2024.
- [21] C. Pavlopoulou, M. J. Carey and V. J. Tsotras, "Revisiting Runtime Dynamic Optimization for Join Queries in Big Data Management Systems," *SIGMOD Rec*, vol. 52, no. 1, pp. 104-113, 2023.
- [22] H. Kalumin and A. Deshpande, "Optimizing Queries with Many-to-Many Joins," in *2024 IEEE 41st International Conference on Data Engineering (ICDE)*, Hong Kong, 2025.
- [23] V. Mageirakos, R. Mancini, S. Karthik, B. Chandra and A. Ailamaki, "Efficient GPU-accelerated Join Optimization for Complex Queries," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, 2022.
- [24] R. Rui, H. Li and Y. C. Tu, "Efficient Join Algorithms For Large Database Tables in a Multi-GPU Environment," in *Proceedings VLDB Endowment*, 2020.

GROUP MEMBER CONTRIBUTION

Table 3: Group Member Contribution Table

Student Name (ID)	Percentage of the overall participation
Wong Hui San (22034540)	100%
Keertana A/P Subramaniam (23109614)	100%
Liu Yong Le (22064141)	100%